

გაკვეთილი 18 Mod 1 (1.4)

გაკვეთილი 6 — განლაგება, 2D ტრანსფორმაცია და ანიმაცია

გაკვეთილის მიზანი

ამ გაკვეთილის შემდეგ მოსწავლემ უნდა შეძლოს:

- ელემენტების ეკრანზე სწორად განლაგება
- `display` თვისების გამოყენება
- `flexbox` -ით ელემენტების ერთ ხაზზე ან სვეტად დალაგება
- `grid` -ით ელემენტების რიგებად და სვეტებად განაწილება
- `transform` -ით ელემენტის გადაადგილება, გადიდება და დატრიალება
- `transition` -ით ეფექტების გლუვად გაკეთება
- `animation` -ით მოძრავი და ცოცხალი ელემენტების შექმნა

1. რას ნიშნავს Layout?

Layout ნიშნავს ვებგვერდზე ელემენტების განლაგებას.

მაგალითად:

- სად დგას მენიუ
- სად არის სურათი
- სად არის ტექსტი
- ელემენტები ერთ ხაზზეა თუ ერთმანეთის ქვემოთ
- როგორ ნაწილდება სივრცე გვერდზე

თუ HTML ქმნის სტრუქტურას, CSS layout თვისებები გვცხმარება ამ სტრუქტურის ლამაზად დალაგებაში.

რეალურ საიტზე layout ძალიან მნიშვნელოვანია, რადგან მხოლოდ ელემენტების არსებობა არ კმარა — საჭიროა, რომ ისინი წესრიგში და სწორ ადგილას იყვნენ.

2. `display` — ელემენტის გამოჩენის ტიპი

`display` განსაზღვრავს, როგორ იქცევა ელემენტი გვერდზე.

ეს ერთ-ერთი ყველაზე მნიშვნელოვანი CSS property-ა, რადგან ბევრი რამ სწორედ აქედან იწყება.

2.1 `display: block`

Block ელემენტი:

- იწყებს ახალ ხაზზე
- იკავებს ხელმისაწვდომ მთელ სიგანეს
- ხშირად გამოიყენება დიდი სტრუქტურული ელემენტებისთვის

მაგალითად, block ელემენტებია:

- `div`
- `p`
- `h1` - `h6`
- `section`
- `article`

მაგალითი

`index.html`

,

First block

Second block

`style.css`

```
.block-box {  
display: block;  
background-color: lightblue;  
border: 1px solid blue;  
margin: 10px 0;  
padding: 20px;  
}
```

რა მოხდება?

ორივე `div` ერთმანეთის ქვემოთ დადგება, რადგან `block` ელემენტები ახალ ხაზზე იწყებენ.

2.2 `display: inline`

Inline ელემენტი:

- არ იწყებს ახალ ხაზზე
- იკავებს მხოლოდ იმდენ ადგილს, რამდენიც სჭირდება
- ტექსტით იქცევა

მაგალითად, `inline` ელემენტებია:

- `span`
- `a`
- `strong`
- `em`

მაგალითი

`index.html`

Inline 1

Inline 2

Inline 3

`style.css`

```
.inline-box {  
display: inline;  
background-color: yellow;  
border: 1px solid orange;  
padding: 10px;  
}
```

რა მოხდება?

სამივე ელემენტი ერთ ხაზზე გამოჩნდება.

2.3 `display: inline-block`

ეს არის block-ის და inline-ის შუალედური ვარიანტი.

`inline-block :`

- ერთ ხაზზე დგება, როგორც inline
- მაგრამ შეგვიძლია მივცეთ `width` , `height` , `margin` , `padding` , როგორც block ელემენტს

ეს ძალიან გამოსადეგია, როცა გვინდა ელემენტები ერთ ხაზზე, მაგრამ ბლოკური კონტროლით.

მაგალითი

`index.html`

Box 1

Box 2

Box 3

`style.css`

```
.inline-block-box {  
display: inline-block;  
width: 120px;  
height: 80px;  
background-color: lightgreen;  
border: 1px solid green;  
margin: 10px;  
padding: 10px;  
}
```

რატომ არის კარგი?

რადგან ელემენტები ერთ ხაზზე არიან, მაგრამ თითოეულს თავისი ზომა აქვს.

2.4 `display: none`

ეს ელემენტს საერთოდ მალავს.

- ეკრანზე არ ჩანს
- ადგილსაც აღარ იკავებს

მაგალითი

style.css

```
.hidden-box {  
display: none;  
}
```

თუ რომელიმე ელემენტს ეს კლასი ექნება, ის გვერდზე საერთოდ აღარ გამოჩნდება.

2.5 როდის რომელი გამოვიყენოთ?

- `block` — დიდი სტრუქტურული ელემენტებისთვის
 - `inline` — ტექსტის შიგნით მყოფი პატარა ელემენტებისთვის
 - `inline-block` — როცა ელემენტები გვერდიგვერდ უნდა დალაგდეს და ზომაც უნდა ჰქონდეს
 - `none` — როცა ელემენტის დამალვა გვინდა
-

3. Flexbox — თანამედროვე ერთგანზომილებიანი განლაგება

3.1 რა არის Flexbox?

Flexbox არის CSS-ის სისტემა, რომელიც გამოიყენება ელემენტების მოქნილად დასალაგებლად.

ძალიან მოსახერხებელია, როცა გვინდა:

- ელემენტები ერთ ხაზზე დავალაგოთ
- ცენტრში მოვათავსოთ
- თანაბარი სივრცე დავუტოვოთ
- სვეტად ან რიგად ვაჩვენოთ

Flexbox განსაკუთრებით კარგია **1 განზომილებაში**:

- ან ჰორიზონტალურად
 - ან ვერტიკალურად
-

3.2 ძირითადი იდეა

Flexbox-ში გვაქვს:

- **parent element** — flex container
- **child elements** — flex items

ანუ მშობელ ელემენტს ვეუბნებით `display: flex;` , და მისი შვილები უკვე flex წესებით ლაგდებიან.

3.3 display: flex

მაგალითი

index.html

```
<html>
<link rel="stylesheet" href="style.css">
<div class="container">
1
<div class="item">2</div>
3
</body>
```

style.css

```
.container {
display: flex;
background-color: #dfe6e9 ;
padding: 20px;
}

.item {
width: 80px;
height: 80px;
background-color: #74b9ff ;
border: 1px solid #0984e3 ;
margin: 10px;
}
```

რა მოხდება?

სამივე `.item` ერთ ხაზზე დადგება.

3.4 flex-direction

ეს property განსაზღვრავს, რა მიმართულებით დალაგდება შვილები.

მნიშვნელობები:

- `row` — მარცხნიდან მარჯვნივ
- `column` — ზემოდან ქვემოთ

მაგალითი

```
.container {  
  display: flex;  
  flex-direction: column;  
}
```

თუ `column` დავუწერეთ, ელემენტები ერთმანეთის ქვემოთ ჩამწკრივდება.

3.5 justify-content

ეს property აკონტროლებს ელემენტების განაწილებას **main axis**-ზე.

თუ `flex-direction: row;` გვაქვს, **main axis** ჰორიზონტალურია.

მნიშვნელობები:

- `flex-start` — დასაწყისში
- `center` — ცენტრში
- `flex-end` — ბოლოს
- `space-between` — ელემენტებს შორის თანაბარი სივრცე
- `space-around` — ელემენტების ირგვლივ სივრცე
- `space-evenly` — ყველგან თანაბარი დაშორება

მაგალითი

```
.container {  
display: flex;  
justify-content: center;  
}
```

შედეგი

ყველა ელემენტი ჰორიზონტალურად ცენტრში დადგება.

3.6 align-items

ეს property აკონტროლებს ელემენტებს **cross axis**-ზე.

თუ `flex-direction: row`; გვაქვს, მაშინ `align-items` ვერტიკალურ გასწორებას აკეთებს.

მნიშვნელობები:

- stretch
- flex-start
- center
- flex-end

მაგალითი

```
.container {  
display: flex;  
align-items: center;  
height: 300px;  
}
```

შედეგი

შვილები container-ის შიგნით ვერტიკალურად ცენტრში დადგებიან.

3.7 gap

Flex ელემენტებს შორის დაშორებას აყენებს.

```
.container {  
display: flex;
```

```
gap: 20px;  
}
```

ეს ბევრად მოსახერხებელია, ვიდრე ყველა item-ზე `margin` დაწერა.

3.8 სრული Flexbox მაგალითი

index.html

Home
About
Contact

style.css

```
.flex-container {  
display: flex;  
justify-content: center;  
align-items: center;  
gap: 20px;  
height: 150px;  
background-color: #f1f2f6 ;  
}  
  
.flex-item {  
width: 100px;  
padding: 15px;  
text-align: center;  
background-color: #a29bfe ;  
border-radius: 8px;  
}
```

3.9 სად გამოიყენება Flexbox?

- navigation bar
- cards ერთ ხაზზე
- ღილაკების განლაგება
- ელემენტის ცენტრში მოთავსება
- პატარა layout-ები

4. Grid — ორგანიზაციული განლაგება

4.1 რა არის Grid?

Grid არის CSS სისტემა, რომელიც ელემენტებს რიგებად და სვეტებად ალაგებს.

თუ Flexbox უფრო ერთ ხაზზე ან ერთ სვეტზე მუშაობს, Grid უფრო ძლიერია მაშინ, როცა გვჭირდება:

- ბევრი ბლოკის ბადედ დალაგება
- რამდენიმე სვეტი
- რამდენიმე რიგი
- უფრო რთული layout

4.2 display: grid

მშობელ ელემენტს ვაძლევთ:

```
.container {  
display: grid;  
}
```

ამის შემდეგ მისი შვილები grid ელემენტებად იქცევიან.

4.3 grid-template-columns

ეს განსაზღვრავს, რამდენი სვეტი იქნება.

მაგალითი

```
.container {  
display: grid;  
grid-template-columns: 1fr 1fr 1fr;  
}
```

ეს ნიშნავს:

- 3 სვეტი
- ყველა თანაბარი სიგანის

4.4 fr ერთეული

fr ნიშნავს **fraction**-ს, ანუ ხელმისაწვდომი სივრცის ნაწილს.

მაგალითად:

```
grid-template-columns: 1fr 2fr 1fr;
```

ეს ნიშნავს:

- პირველი სვეტი — 1 ნაწილი
- მეორე სვეტი — 2 ნაწილი
- მესამე სვეტი — 1 ნაწილი

ანუ მეორე უფრო ფართო იქნება.

4.5 gap

Grid ელემენტებს შორის დაშორებას აყენებს.

```
.container {  
  display: grid;  
  grid-template-columns: 1fr 1fr 1fr;  
  gap: 15px;  
}
```

4.6 სრული Grid მაგალითი

index.html

```
1  
2  
3  
4  
5  
6
```

style.css

```
.grid-container {  
display: grid;  
grid-template-columns: 1fr 1fr 1fr;  
gap: 15px;  
padding: 20px;  
background-color: #ecf0f1 ;  
}
```

```
.grid-box {  
background-color: #fab1a0 ;  
padding: 30px;  
text-align: center;  
border-radius: 10px;  
}
```

რა მოხდება?

ელემენტები 3 სვეტად განლაგდება.

Order - ძირითადი იდეა

ნორმალურად ელემენტები მოდიან იმ რიგით, როგორც HTML-ში წერია
მაგრამ `order` -ით შეგიძლია უთხრა:

👉 „ეს ელემენტი წინ იყოს“ ან „უკან წავიდეს“

◆ სინტაქსი

```
.item {  
order: number;  
}
```

- default მნიშვნელობა: `order: 0`
 - რაც მცირე რიცხვია → უფრო ადრე გამოჩნდება
 - რაც დიდია → უფრო გვიან
-

◆ მარტივი მაგალითი

HTML:

A
B
C

CSS:

```
.container {  
display: flex;  
}
```

```
.a {  
order: 2;  
}
```

```
.b {  
order: 1;  
}
```

```
.c {  
order: 3;  
}
```

◆ შედეგი:

მიუხედავად იმისა, რომ HTML-ში არის:

A B C

ეკრანზე გამოვა:

B A C

◆ flex-wrap:

`flex-wrap` არის Flexbox-ის property, რომელიც განსაზღვრავს:

👉 ელემენტები ერთ ხაზში დარჩებიან თუ გადავლენ ახალ ხაზზე

◆ ძირითადი იდეა

Flexbox-ში default ქცევა არის:

👉 ყველა ელემენტი ერთ ხაზში ეტევა (თუნდაც დაპატარავდეს)

```
.container {  
display: flex;  
flex-wrap: nowrap; / default /  
}
```

◆ flex-wrap მნიშვნელობები

1. nowrap (default)

👉 არ გადადის ახალ ხაზზე

```
.container {  
display: flex;  
flex-wrap: nowrap;  
}
```

📌 შედეგი:

- ყველა ელემენტი ერთ ხაზში
 - შეიძლება ძალიან დაპატარავდეს 😬
-

2. wrap

👉 არ ეტევა? → გადადის ახალ ხაზზე

```
.container {  
display: flex;  
flex-wrap: wrap;  
}
```

📌 შედეგი:

```
[ item ][ item ][ item ]  
[ item ][ item ]
```

👉 როგორც ტექსტი, როცა ხაზს ცდება

3. wrap-reverse

👉 იგივე wrap , მაგრამ ხაზები პირიქით

```
.container {  
display: flex;  
flex-wrap: wrap-reverse;  
}
```

📌 შედეგი:

- ახალი ხაზი ქვემოთ კი არა → ზემოთ იწყება

4.7 Grid vs Flexbox

Flexbox

საუკეთესოა:

- ერთ ხაზზე განლაგებისთვის
- პატარა კომპონენტებისთვის
- მენიუსთვის
- ბარათების ერთ ხაზზე დასალაგებლად

Grid

საუკეთესოა:

- დიდი გვერდის layout-ისთვის
- რიგებისა და სვეტებისთვის
- გალერეებისთვის
- dashboard-ებისთვის

5. Transform — ელემენტის გარდაქმნა

5.1 რა არის transform ?

transform გამოიყენება ელემენტის ვიზუალური ცვლილებისთვის, ისე რომ HTML სტრუქტურა არ იცვლება.

მას შეუძლია:

- გადაადგილება
- გაზრდა/დაპატარავება
- დატრიალება
- დახრა

ჩვენს თემაში მთავარი არის **2D transform**.

5.2 `translate()`

ელემენტს გადაადგილებს.

```
.box {  
transform: translate(50px, 20px);  
}
```

ეს ნიშნავს:

- მარჯვნივ 50px
- ქვემოთ 20px

თუ მხოლოდ X ღერძზე გვინდა:

```
transform: translateX(50px);
```

თუ მხოლოდ Y ღერძზე:

```
transform: translateY(30px);
```

5.3 `scale()`

ელემენტს ადიდებს ან აპატარავებს.

```
.box {  
transform: scale(1.2);  
}
```

ეს ნიშნავს, რომ ელემენტი 20%-ით გაიზრდება.

```
transform: scale(0.8);
```

ეს ნიშნავს, რომ ელემენტი შემცირდება.

5.4 rotate()

ელემენტს ატრიალებს.

```
.box {  
transform: rotate(45deg);  
}
```

ეს ნიშნავს 45 გრადუსით ბრუნვას.

5.5 transform-ის პრაქტიკული გამოყენება

ძალიან ხშირად გამოიყენება hover ეფექტებზე.

მაგალითად:

```
.card:hover {  
transform: scale(1.05);  
}
```

ან

```
.button:hover {  
transform: translateY(-5px);  
}
```

ან

```
.icon:hover {  
transform: rotate(10deg);  
}
```

5.6 Transform სრული მაგალითი

index.html

Hover me

style.css

```
.transform-box {  
width: 200px;  
padding: 30px;  
margin: 50px auto;  
text-align: center;  
background-color: #55efc4 ;  
border-radius: 10px;  
}  
  
.transform-box:hover {  
transform: scale(1.1);  
}
```

6. Transition — გლუვი ეფექტი

6.1 რა არის Transition?

თუ hover-ზე ელემენტს ფერი ან ზომა უცებ ეცვლება, მოძრაობა მკვეთრია. `transition` ამ ცვლილებას გლუვს ხდის.

ანუ state-ის ცვლილება ხდება ნელა და ლამაზად.

6.2 მარტივი სინტაქსი

```
.box {  
transition: 0.3s;  
}
```

ეს ნიშნავს, რომ ყველა შეცვლადი property 0.3 წამში შეიცვლება.

6.3 სრული სინტაქსი

transition: property duration timing-function delay;

მაგალითად:

transition: transform 0.4s ease;

ეს ნიშნავს:

- რომელი property იცვლება → `transform`
 - რამდენ ხანში → `0.4s`
 - რა სტილით → `ease`
-

6.4 ყველაზე გავრცელებული timing-function-ები

- `ease`
 - `linear`
 - `ease-in`
 - `ease-out`
 - `ease-in-out`
-

6.5 მაგალითი — ფერის ცვლილება

index.html

Hover me

style.css

```
.btn {  
background-color: #0984e3 ;  
color: white;  
border: none;  
padding: 15px 25px;  
border-radius: 8px;  
transition: background-color 0.4s ease;  
}  
  
.btn:hover {  
background-color: #d63031 ;  
}
```

6.6 მაგალითი — transform + transition

```
.card {  
transition: transform 0.3s ease, box-shadow 0.3s ease;
```

```
}  
  
.card:hover {  
transform: scale(1.05);  
box-shadow: 0 10px 25px rgba(0, 0, 0, 0.2);  
}
```

7. Animation — ავტომატური მოძრაობა

7.1 რა არის Animation?

`animation` საშუალებას გვაძლევს ელემენტი თვითონ, ავტომატურად იმოძრაოს ან შეიცვალოს.

Transition-ისგან განსხვავებით:

- `transition` ჩვეულებრივ action-ზე რეაგირებს (`hover`, `focus`, `click`)
 - `animation` თავისითაც შეიძლება დაიწყოს
-

7.2 @keyframes

`@keyframes` აღწერს ანიმაციის ნაბიჯებს.

ანუ ვამბობთ:

- თავიდან რა მდგომარეობაშია
- ბოლოს რა მდგომარეობაშია

ან შეიძლება შუაშიც მივუთითოთ.

7.3 მარტივი მაგალითი

```
@keyframes moveBox {  
from {  
transform: translateX(0);  
}  
to {  
transform: translateX(200px);  
}
```

```
}  
}
```

ეს ნიშნავს:

- თავიდან ელემენტი ჩვეულ ადგილას არის
 - ბოლოს 200px-ით მარჯვნივ გადადის
-

7.4 animation property

```
.box {  
  animation: moveBox 2s infinite;  
}
```

ეს ნიშნავს:

- გამოიყენე `moveBox`
 - ხანგრძლივობა 2 წამი
 - უსასრულოდ განმეორდეს
-

7.5 უფრო სრული ფორმა

animation: name duration timing-function delay iteration-count direction;

მაგალითად:

animation: moveBox 2s ease-in-out 0s infinite alternate;

7.6 მნიშვნელოვანი მნიშვნელობები

iteration-count

რამდენჯერ განმეორდეს:

- 1
- 2
- `infinite`

direction

რომელი მიმართულებით წავიდეს:

- `normal`
- `reverse`
- `alternate`

`alternate` ნიშნავს წინ და უკან მოძრაობას.

7.7 სრული Animation მაგალითი

`index.html`

`style.css`

```
.animated-box {  
width: 100px;  
height: 100px;  
background-color: crimson;  
border-radius: 12px;  
animation: moveRight 2s infinite alternate;  
}
```

```
@keyframes moveRight {  
from {  
transform: translateX(0);  
}  
to {  
transform: translateX(250px);  
}  
}
```

7.8 მრავალნაბიჯიანი ანიმაცია

შეიძლება `0%`, `50%`, `100%` ფორმითაც დავწეროთ.

```
@keyframes colorMove {  
0% {  
transform: translateX(0);  
background-color: red;  
}
```

```
50% {  
transform: translateX(100px);  
background-color: blue;  
}  
  
100% {  
transform: translateX(200px);  
background-color: green;  
}  
}
```

8. Transition და Animation შორის განსხვავება

Transition

- საჭიროა trigger, მაგალითად `hover`
- ერთი მდგომარეობიდან მეორეზე გადასვლისთვისაა

Animation

- შეიძლება თავისით დაიწყოს
 - რამდენიმე ნაბიჯიც ჰქონდეს
 - განმეორებადიც შეიძლება იყოს
-

9. პრაქტიკული დიდი მაგალითი — Card Layout + Hover Effect

`index.html`

Card 1

Simple hover effect with transform and transition.

Card 2

Cards are arranged using flexbox.

Card 3

This is a modern UI example.

style.css

```
.cards {  
display: flex;  
justify-content: center;  
gap: 20px;  
padding: 40px;  
}  
  
.card {  
width: 250px;  
padding: 20px;  
background-color: #f1f2f6;  
border-radius: 16px;  
box-shadow: 0 6px 18px rgba(0, 0, 0, 0.15);  
transition: transform 0.3s ease, box-shadow 0.3s ease;  
}  
  
.card:hover {  
transform: translateY(-8px) scale(1.03);  
box-shadow: 0 14px 28px rgba(0, 0, 0, 0.2);  
}
```

10. დიდი Grid მაგალითი — პატარა გალერეა

index.html

1
2
3
4

5

6

style.css

```
.gallery {  
display: grid;  
grid-template-columns: 1fr 1fr 1fr;  
gap: 20px;  
padding: 30px;  
}
```

```
.gallery-item {  
background-color: #ffeaa7 ;  
padding: 50px 20px;  
text-align: center;  
border-radius: 12px;  
transition: transform 0.3s ease;  
}
```

```
.gallery-item:hover {  
transform: scale(1.05);  
}
```

11. რეალურ პროექტებში სად ვიყენებთ ამ ყველაფერს?

display

- ელემენტის ტიპის შესაცვლელად
- მენიუს ელემენტების დასალაგებლად

flexbox

- ნავიგაცია
- card-ები
- ღილაკების განლაგება
- ელემენტის ცენტრში მოთავსება

grid

- gallery

- dashboard
- პროდუქტის ბადე
- სრული გვერდის layout

transform

- hover ეფექტები
- ღილაკის მოძრაობა
- ბარათის გადიდება
- სურათის ოდნავ დატრიალება

transition

- smooth hover
- ფერის ნელი ცვლილება
- ზომის გლუვი ცვლილება

animation

- მბრუნავი ელემენტები
- მოძრაობა
- loading ეფექტები
- ცოცხალი დიზაინი

12. ხშირი შეცდომები

1. display: flex child-ზე წერა

Flexbox უნდა დაეწეროს **parent** ელემენტს და არა შვილებს.

2. justify-content და align-items არევა

- justify-content — main axis
- align-items — cross axis

3. transform -ის მოულოდნელი მკვეთრი ეფექტი

თუ smooth გინდა, transition უნდა დაამატო.

4. animation-ის სახელის და keyframes-ის სახელის ართხვევა

თუ `animation: moveBox 2s;` წერ, `keyframes`-იც `moveBox` უნდა ერქვას.

5. grid-ზე `grid-template-columns` არ დაწერა

თუ სვეტები არ განსაზღვრე, შედეგი ისეთი არ იქნება, როგორსაც ელოდი.

13. სავარჯიშოები

მარტივი

1.

შექმენი 3 `div` და დაალაგე ერთ ხაზზე `display: flex;` გამოყენებით.

2.

იგივე ბლოკები დაალაგე სვეტად `flex-direction: column;`-ით.

3.

შექმენი 4 ბლოკი `grid`-ით 2 სვეტად.

4.

შექმენი 1 ბლოკი, რომელიც `hover`-ზე გაიზრდება `scale()`-ით.

5.

შექმენი ღილაკი, რომელსაც `hover`-ზე ფერი გლუვად შეეცვლება.

საშუალო

6.

შექმენი navigation bar, სადაც 4 ელემენტი `flexbox`-ით იქნება ერთ ხაზზე და ცენტრში.

7.

შექმენი 6 ბლოკიანი gallery `grid`-ით.

8.

შექმენი card, რომელიც hover-ზე:

- ოდნავ აიწევს
- ოდნავ გაიზრდება
- ჩრდილი მოემატება

9.

შექმენი კვადრატი, რომელიც ეკრანზე მარჯვნივ წავა animation -ით.

10.

შექმენი 3 card flexbox-ით, რომლებსაც ერთმანეთში თანაბარი დამორება ექნებათ.

რთული

11.

შექმენი პატარა landing page section:

- ზედა სათაური
- ქვემოთ 3 card
- card-ები flexbox-ით
- hover-ზე transform + transition

12.

შექმენი product grid:

- 6 პროდუქტი
- 3 სვეტი
- თითოეულს ჰქონდეს hover ეფექტი

13.

შექმენი ანიმირებული ღილაკი, რომელიც hover-ზე:

- იზრდება
- ფერს იცვლის
- ჩრდილს იმატებს

14.

შექმენი ელემენტი, რომელიც უსასრულოდ მოძრაობს წინ და უკან `alternate` -ით.

15.

შექმენი გვერდი, სადაც ერთად გამოიყენებ:

- `display`
 - `flexbox`
 - `grid`
 - `transform`
 - `transition`
 - `animation`
-

ძალიან რთული

16.

შექმენი “Student Dashboard” დიზაინი:

- `header`
- მენიუ `flexbox`-ით
- `cards grid`-ით
- `hover` ეფექტები
- ერთი ანიმირებული ბლოკი

17.

შექმენი “Modern Portfolio Section”:

- მთავარი ტექსტი
- პროექტების ბადე
- `card hover animation`
- ერთი მოძრავი დეკორატიული ბლოკი

18.

შექმენი პატარა ონლაინ მაღაზიის მთავარი ნაწილი:

- ზედა ნავიგაცია `flexbox`-ით
- პროდუქტები `grid`-ით
- პროდუქტის `card hover`-ზე იზრდებოდეს

- ღილაკს ჰქონდეს smooth transition